

本 demo 只作为企业微信开放数据 API 的学习和演示，不建议在生产环境上使用。
企业微信产研团队不对 demo 的使用负责。

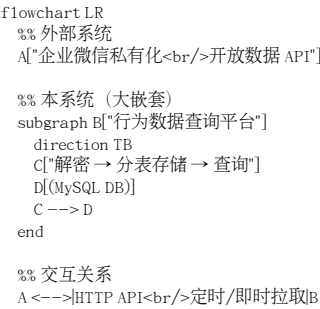
开放源代码 [Gitee 链接](#)

企业微信私有化开放数据获取与查询demo

1. 系统概述

1.1 背景与定位

本软件是面向 企业微信私有化部署 场景的日志数据自动获取与查询的demo。企业微信私有化提供了一套「开放数据 API」，能够输出覆盖用户行为、系统运行、应用交互、会话内容等维度的运营日志数据——但这些数据以加密形式返回，且接口有分页限制和时效窗口约束。



1.2 核心价值

能力维度	描述
自动化采集	支持定时（可配频率）和即时两种模式，自动分页拉取、失败重试、token 自动刷新
安全解密	RSA + AES-128-CBC 双层解密引擎，支持多密钥版本热切换
高效存储	按 FeatureID + 月份 自动分表，每种数据项拥有独立的表结构和索引
灵活查询	支持按数据项、时间范围、动态条件查询，支持跨月汇总统计与多粒度图表
行为审计	可按用户 OpenID 跨 24 种数据类型聚合查询完整行为轨迹
平台干预	直接调用企微私有化 API 执行撤回消息、删除消息/文件、解散群、踢人等干预操作
三权分立	系统管理员、配置管理员、审计管理员三种角色隔离，职责清晰
运维友好	单一二进制部署、YAML 配置、CLI 工具、操作日志审计、自动数据维护

2. 功能全景

2.1 数据采集引擎

系统提供两种互补的数据采集模式：

定时任务采集

配置管理员可设定数据拉取频率，系统基于 robfig/cron 引擎自动执行：

- 每小时一次：可指定每小时的第几分钟触发。starttime/endtime 对应上一整小时的区间
- 每 N 分钟一次：可选间隔 2/3/5/6/10/12/15/20/30 分钟。时间窗口自动向前取整对齐
- 可勾选需要采集的数据项编码（24 种任意组合）
- 运行状态下可查看下次执行时间与倒计时
- 服务重启后自动恢复上次运行状态

即时任务采集

当需要补拉历史数据或重新采集某时间段数据时：

- 指定起止时间，按小时粒度拆分循环拉取
- 时间跨度限制：最小 1 小时，最大 7 天
- 并发互斥：同一时间只允许运行一个即时任务
- 事务性写入：每个小时窗口内，先删除旧数据再插入新数据（DELETE + INSERT 事务）
- 实时进度：前端可查看"正在处理 YYYY-MM-DD HH 的数据拉取"
- 失败即停：任何 API 调用或数据库写入失败立即终止整个任务并记录错误

API 调用策略

策略 实现

分页拉取 limit=1000，自动翻页直到返回空数组或数量不足
失败重试 最多 3 次，每次间隔 1 分钟
Token 管理 内存缓存，过期前 5 分钟自动刷新，使用 corpid + corpsecret 获取
跨天处理 当时间窗口跨天时自动拆分为两段（API 要求同一天内）

2.2 数据解密与存储

两级解密引擎

从企业微信 API 返回的日志数据是加密的。系统实现了完整的解密流水线：

enc_key (Base64) → RSA PKCS1v15 解密 → AES Key
enc_data (Base64) → 提取 IV (前16字节) → AES-128-CBC 解密 → PKCS7 去填充 → 截去尾部8字节 → JSON 明文

- 支持按 enc_ver （密钥版本号）索引对应的 RSA 私钥
- 支持多密钥版本并存，可通过管理界面随时添加新版本私钥
- 私钥以 PEM 格式明文存储在数据库中

动态分表存储

解密后的 JSON 数据经过字段映射（25 种独立的映射函数），写入按 featureid + 月份 命名的分表：

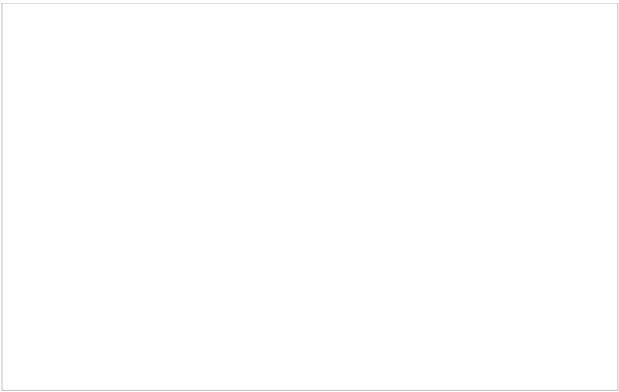
表名规则：log_{featureid}_{YYYYMM}
示例：log_90000031_202603 （2026年3月的登录日志）

- 每种 featureID 拥有独立的 DDL 列定义（共 25 种不同的表结构）
- 分表在数据写入时按需自动创建，无需手动建表
- 支持跨月查询时自动聚合多张分表

2.3 数据查询与审计

审计管理员和配置管理员均可使用数据查询功能：

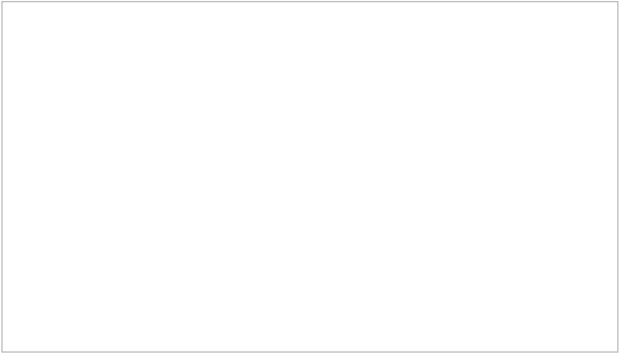
- 按数据项查询：选择 featureID + 时间范围，系统自动定位对应分表
- 动态条件过滤：支持 =、!=、>、<、>=、<=、like、not_like 等操作符
- 跨月查询：自动拼接多张分表数据，统一分页返回
- 多粒度统计：按分钟 小时 天粒度统计日志条数，图表可视化展示
- 分页浏览：表格列根据不同数据项自动适配



2.4 用户行为追踪

审计管理员可通过「行为查询」功能，输入用户的 OpenID，跨 24 种数据类型聚合查询该用户的完整行为轨迹：

- **并发查询**：使用 goroutine 并发查询多个 featureID 的分表
- **智能字段匹配**：自动识别每种数据类型中的 openid 字段位置（单值字段 / JSON 数组字段）
- **去重处理**：基于数据库 ID 和内容双重去重
- **结果排序**：按时间正序排列，支持按特定数据类型过滤
- **命中高亮**：标记每条记录中命中查询条件的具体字段



2.5 平台干预操作

审计管理员可直接通过平台对企业微信用户和群聊执行干预操作。系统封装了以下企微私有化 API：

干预操作	企微 API 端点	说明
撤回消息	/cgi-bin/corp/revoke_msg	支持无痕撤回（revokemsg_notrace=1）
删除消息	/cgi-bin/corp/del_msg	按 msgid 删除指定消息
删除文件	/cgi-bin/corp/del_msg_file	按 fileid 删除消息附件
解散群聊	/cgi-bin/corp/dismiss_room	指定 chatid 和操作人解散群
移除群成员	/cgi-bin/corp/del_room_member	批量移除指定群成员

所有干预操作均记录到操作日志表，支持事后审计。

2.6 权限与安全管理

系统采用 三权分立 的 RBAC 模型，一个用户只能拥有一种管理员身份：

角色	代码	职责范围	可访问功能
系统管理员	role=1	管理所有管理员账号	用户管理、维护配置、数据统计、任务记录、全局操作日志
配置管理员	role=2	配置和执行数据采集	定时任务配置、即时任务、私钥管理、数据统计、数据查询
审计管理员	role=3	审计与干预	数据查询、行为查询、干预操作、数据统计

认证机制

- **Session + Cookie**：使用内存 SessionStore，24 小时过期，每 10 分钟后台清理
- **密码加盐哈希**：bcrypt(salt + password + salt)，盐值可通过配置文件自定义
- **双密码设计**：未激活账号使用 init_password_hash，激活后使用 password_hash

- **密码复杂度**：最少 8 字符，必须包含大写字母 + 小写字母 + 数字 + 特殊字符
- **防暴力破解**：连续 5 次输错密码锁定账号 10 分钟
- **修改密码踢出**：密码修改后自动删除该用户所有 Session，强制重新登录

2.7 系统运维管理

操作日志

系统记录所有关键操作，用于安全审计和追溯：

操作类型 记录内容

登录日志 登录时间、来源 IP、登录结果（成功/失败/锁定/禁用）
配置变更 定时任务频率修改、数据项变更
任务操作 启动/停止定时任务、创建即时任务
用户管理 创建/禁用/启用/删除管理员、重置密码
干预操作 撤回消息、删除消息/文件、解散群、移除群成员

- 系统管理员可查看所有人的操作日志（支持按账号、操作类型、时间范围过滤）
- 配置管理员和审计管理员只能查看自己的操作日志

数据维护

系统管理员可配置自动数据维护策略：

- **日志保留期**：可设置日志分表保留月数（默认 6 个月），超期自动 Drop 整表
- **任务记录保留期**：可设置 task_records 保留天数（默认 30 天）
- **维护时间**：可设定每日执行时间（默认 05:00）
- **统计缓存**：维护任务执行时自动统计各分表数据量

3. 支持的数据类型

本系统覆盖企业微信私有化开放的 4 大类 24 + 1 种日志数据项：

数据大类	数据项编码	数据说明	主要字段
成员使用日志	90000031	登录	openid, deviceid, login_time, cli_ver, access_ip
	90000032	唤醒	openid, deviceid, login_time, cli_ver
	90000035	聊天消息发送	sender, chat_type, msg_type, send_time
	90000038	创建群聊	creator, chatid, name, members
	90000039	群加入	oper, add_members, chatid
	90000040	群踢人	oper, del_members, chatid
	90000041	退群	quit_user, chatid, original_members
	90000042	转让群主	oper, new_owner, chatid
	90000043	解散群	oper, chatid, members
	90000044	群改名	oper, old_name, new_name
	90000047	添加	user, operate_time
	90000048	激活	user, operate_time
	90000054	客户端安装信息	openid, deviceid, cli_ver, timestamp
	90000055	客户端更新信息	openid, old_ver, new_ver
	90000064	点击导航	openid, tab_type, access_time
	90000065	接收文件	receiver, chat_type, receive_time
系统日志	90000056	消息列表拉取异常	openid, failed_reason, net_status
	90000057	用户操作失败	openid, event_type, failed_reason
	90000058	通讯录更新	openid, update_type, update_result
	90000059	客户端网络请求统计	openid, req_count, req_avg_cost
应用数据	90000033	访问应用	user, agentid, access_time
	90000034	应用推送消息	agentid, push_type, to_user
聊天数据	90000036	单聊聊天数据	sender, receiver, msg_type, msgid
	90000037	群聊聊天数据	sender, receiver[], chatid, msgid
	90000046	用户与应用聊天数据	user, agentid, send_target, msg_type

4. 软件架构

4.1 架构概览

本系统采用经典的 前后端分离 + 分层架构 设计，后端为单体应用（Go 二进制），前端为 SPA 单页应用（Vue 3），编译后内嵌于后端静态文件服务中，实现 单一进程部署。

4.2 技术栈

层次	技术选型	版本	说明
后端语言	Go	≥ 1.26	高性能，原生并发支持
Web 框架	Gin	v1.12	轻量高效的 HTTP 框架
ORM	GORM	v1.31	数据库操作与自动迁移
数据库	MySQL	≥ 5.7 / 8.0	主要数据存储
任务调度	robfig/cron v3	v3.0.1	定时任务引擎
配置管理	Viper	v1.21	YAML 配置文件读取
CLI 工具	Cobra	v1.10	命令行管理工具
加密	Go 标准库	-	RSA PKCS1v15 + AES-128-CBC
密码哈希	bcrypt	-	加盐哈希
前端框架	Vue 3 + TypeScript 3.x		SPA 单页应用
UI 组件库	Element Plus	最新版	后台管理 UI 组件
状态管理	Pinia	最新版	Vue 3 官方推荐
HTTP 客户端	Axios	最新版	前端 API 请求
构建工具	Vite	最新版	快速前端构建

4.3 系统架构图



MySQL DB	企业微信私有化
	开放数据 API 服务
admins	/cgi-bin/corp/
task_configs	get_log_list
task_records	revoke_msg
private_keys	del_msg
operation_logs	dismiss_room ...
maintenance_*	
log_*.YYYYMM	
(动态分表x25种)	

4.4 分层架构详解

层次	目录	职责	关键文件
入口层	cmd/	应用启动、依赖注入、组件组装	server/main.go（Web 服务）、cli/main.go（CLI 工具）
路由层	internal/handler/	HTTP 路由注册、请求参数解析、响应封装	router.go（路由表）、各业务 handler（10 个）
中间件层	internal/middleware/	认证鉴权、Session 管理	session.go（认证+角色中间件）、session_store.go（内存 Session 存储）
业务逻辑层	internal/service/	业务编排、规则校验	10 个 service 文件
数据访问层	internal/repository/	GORM 数据库操作、分表管理	log_repo.go（核心，843行，含动态建表+跨月查询）
任务调度层	internal/task/	定时/即时任务执行、数据字段映射	scheduler.go、instant_task.go、mapping.go（25种映射）
API 客户端层	internal/api/	企微 API 交互、Token 管理	client.go（数据拉取+干预操作）、token.go
加密层	internal/crypto/	RSA + AES 解密	decrypt.go
模型层	internal/model/	数据结构定义、常量	10 个模型文件（含 25 种日志结构体）
配置层	internal/config/	YAML 配置加载	config.go（基于 Viper）
公共工具	pkg/	分表命名、密码哈希、响应格式	table_naming.go、hash.go、response.go
前端	web/src/	Vue 3 SPA 应用	19 个 .vue 页面组件、路由守卫、Pinia 状态管理

4.5 项目目录结构

wwllogdatacrawler/	
├── cmd/	# 程序入口
│ ├── server/main.go	# Web 服务主入口（依赖注入+优雅关闭）
│ └── cli/main.go	# CLI 管理工具（重置密码、禁用账号）
├── config.yaml	# 配置文件
├── internal/	# 内部业务逻辑（不对外暴露）
│ ├── api/	# 企微 API 客户端
│ │ ├── client.go	# 数据拉取（分页+重试）+ 干预操作
│ │ └── token.go	# access_token 自动获取/刷新
│ ├── config/	# 配置加载
│ │ ├── config.go	# Viper + YAML
│ ├── crypto/	# 数据解密引擎
│ │ └── decrypt.go	# RSA + AES-128-CBC 两级解密
│ ├── handler/	# HTTP 请求处理器（10 个文件）
│ │ ├── router.go	# 路由注册（含 SPA 静态文件、角色分组）
│ │ ├── auth_handler.go	# 认证（登录/登出/改密码）
│ │ ├── admin_handler.go	# 管理员 CRUD
│ │ ├── task_handler.go	# 任务配置与执行
│ │ ├── data_query_handler.go	# 数据查询与统计
│ │ ├── behavior_query_handler.go	# 用户行为查询
│ │ ├── intervention_handler.go	# 干预操作
│ │ ├── private_key_handler.go	# 私钥管理
│ │ ├── maintenance_handler.go	# 维护配置
│ │ └── log_statistics_handler.go	# 统计数据
│ ├── middleware/	# 中间件
│ │ ├── session.go	# SessionAuth + RequireRole + RequireAnyRole
│ │ ├── session_store.go	# 内存 Session 存储（24h 过期，10min 清理）
│ │ └── session_store_test.go	# 单元测试
│ ├── model/	# 数据模型（10 个文件）
│ │ ├── admin.go	# 管理员（3 角色 × 3 状态 × 双密码哈希）
│ │ ├── feature.go	# 25 种数据项编码常量+名称映射
│ │ ├── log_models.go	# 25 种日志结构体模型
│ │ ├── task_config.go	# 定时任务配置
│ │ ├── task_record.go	# 任务执行记录
│ │ ├── private_key.go	# RSA 私钥
│ │ ├── operation_log.go	# 操作日志
│ │ ├── intervention.go	# 干预操作请求/响应模型
│ │ ├── behavior_query.go	# 用户行为查询（含 24 种 openid 字段映射）
│ │ ├── maintenance_config.go	# 维护配置
│ │ └── log_statistics.go	# 统计缓存
│ ├── repository/	# 数据访问层（8 个文件）
│ │ ├── log_repo.go	# ★ 核心：动态分表建表+跨月查询+行为查询（843行）
│ │ ├── admin_repo.go	# 管理员 CRUD
│ │ ├── task_config_repo.go	# 任务配置
│ │ ├── task_record_repo.go	# 任务记录
│ │ ├── private_key_repo.go	# 私钥
│ │ ├── operation_log_repo.go	# 操作日志
│ │ ├── maintenance_config_repo.go	# 维护配置
│ │ └── log_statistics_repo.go	# 统计缓存
│ ├── service/	# 业务逻辑层（10 个文件）
│ │ ├── auth_service.go	# 认证（登录+防暴力破解+密码复杂度）
│ │ ├── admin_service.go	# 管理员管理
│ │ ├── task_service.go	# 任务编排（定时+即时+进度查询）
│ │ ├── data_query_service.go	# 数据查询
│ │ └── behavior_query_service.go	# 行为查询

```
| | | | | intervention_service.go # 干预操作 (撤回/删除/解散/踢人)
| | | | | private_key_service.go # 私钥管理
| | | | | operation_log_service.go # 操作日志
| | | | | maintenance_service.go # 维护服务
| | | | | log_statistics_service.go # 统计服务
| | | | | task/ # 任务执行引擎
| | | | | | scheduler.go # 定时任务调度器 (cron + 维护任务调度)
| | | | | | instant_task.go # 即时任务执行器 (互斥+进度+事务写入)
| | | | | | mapping.go # 25 种 FeatureID 的 JSON→DB 字段映射
| | | | | | maintenance_task.go # 数据维护 (Drop 过期表+统计+清理记录)
| | | | | migrations/ # 数据库迁移脚本
| | | | | | init.sql # 初始建表 (6 张系统表)
| | | | | | add_maintenance_tables.sql # 维护管理表 (2 张)
| | | | | pkg/ # 公共工具包
| | | | | | hash.go # bcrypt 哈希、密码复杂度校验、Session Token
| | | | | | response.go # 统一 API 响应格式
| | | | | | table_naming.go # 分表命名、月份/小时范围、保留期计算
| | | | | web/ # 前端源码
| | | | | | src/
| | | | | | | api/ # 11 个 API 调用模块
| | | | | | | components/ # 公共组件 (StatsChart 图表)
| | | | | | | constants/ # 数据项配置、列定义
| | | | | | | router/index.ts # 路由 (角色守卫+权限控制)
| | | | | | | stores/ # Pinia (user + labels)
| | | | | | | utils/ # Axios 封装、日期工具
| | | | | | | views/ # 19 个页面组件
| | | | | | | | login/ # 登录页 + 改密页
| | | | | | | | layout/ # 主布局 (侧边栏+顶栏)
| | | | | | | | sysadmin/ # 系统管理员 (5 页面)
| | | | | | | | configadmin/ # 配置管理员 (5 页面)
| | | | | | | | audadmin/ # 审计管理员 (3 页面)
| | | | | | | | common/ # 通用组件 (操作日志)
| | | | | | package.json
| | | | | scripts/
| | | | | | run_tests.sh # 自动化测试脚本
| | | | | tests/ # 测试文件
| | | | | Docs/ # 开发文档
| | | | | Makefile # 构建命令
| | | | | go.mod / go.sum # Go 依赖管理
| | | | | LICENSE
```

4.6 数据库设计

系统表（8 张）

表名	说明	关键字段
admins	管理员信息	username, role(1/2/3), status(0/1/2), password_hash, init_password_hash, login_fail_count
task_configs	定时任务配置	frequency_type(1:时/2:分), minute_offset, interval_minutes, feature_ids(JSON), is_running
task_records	任务执行记录	task_type(1:定时/2:即时), status(0:运行/1:完成/2:失败), current_progress, total_hours, finished_hours
private_keys	RSA 私钥存储	enc_ver(唯一), pem_content(PEM 格式)
operation_logs	操作审计日志	operator_id, op_type(1:登录/2:配置/3:任务/4:用户/5:干预), description, op_ip, result
token_cache	access_token 缓存	access_token, expires_at
maintenance_config	维护配置	log_retention_months, task_record_retention_days, maintenance_time
log_statistics	统计缓存	feature_id, month, record_count

日志分表（动态创建，最多 25 × N 张）

命名规则: log_{featureid}_{YYYYMM}

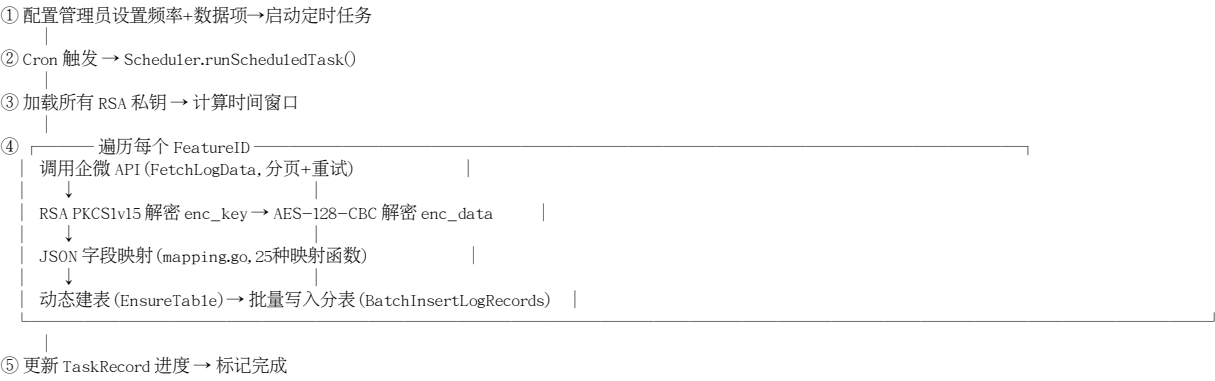
- 每种 featureID 拥有独立的列定义（25 种不同 Schema）
- 表在数据写入时按需自动创建（EnsureTable）
- 公共字段: id, feature_id, log_time
- 特有字段: 根据数据类型不同而异（如登录表有 login_user_openid, cli_ver, access_ip 等）

ER 关系概览

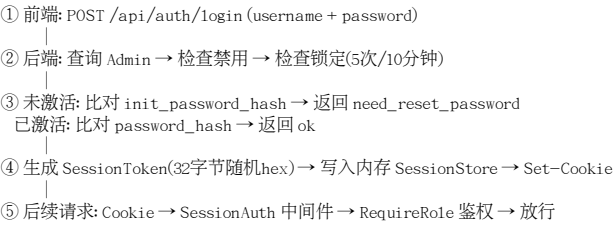
```
admins —(1:N)→ operation_logs (操作人 → 操作日志)
admins —(1:1)→ task_configs (配置管理员 → 任务配置)
admins —(1:N)→ task_records (创建人 → 任务记录)
admins —(1:N)→ private_keys (创建人 → 私钥记录)
task_configs → task_records (配置 → 执行记录)
private_keys → log_{fid}_{month} (密钥 → 解密后数据)
```

4.7 核心数据流

定时数据拉取流程



用户认证流程



4.8 关键设计决策

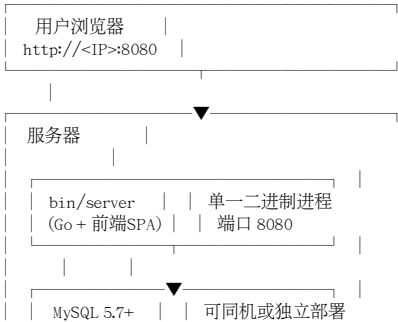
决策	方案	理由
Session 方案	内存 SessionStore	无外部 Redis 依赖，部署简单。改密后可精确踢出指定用户所有 Session
数据分表	log_{fid}_{YYYYMM} 动态分表	25 种数据类型有不同的列定义，按月分表控制单表数据量（10万~100万级）
建表方式	原生 SQL + information_schema 检测	GORM AutoMigrate 不适用于动态多 Schema 场景
即时任务互斥	sync.Mutex + currentTask 单例	防止并发执行导致数据覆盖冲突
即时任务写入	事务性 DELETE + INSERT	保证数据一致性，支持重新采集覆盖旧数据
加密方案	RSA PKCS1v15 + AES-128-CBC + PKCS7 + 尾部8字节截断	与企微官方 Java SDK 保持一致
前端 SPA 部署	编译后内嵌于 Gin 静态文件服务	单进程部署，NoRoute fallback 支持 History 模式路由
配置管理	Viper + YAML	支持默认值、环境变量覆盖、热加载
依赖注入	手动注入（main.go 组装）	项目规模适中，避免引入 DI 框架增加复杂度

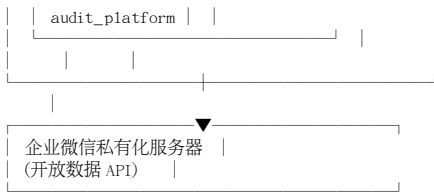
5. 部署指南

5.1 环境要求

组件	版本要求	说明
Go	≥ 1.26	编译后端
Node.js	≥ 18（推荐 20+）	编译前端
MySQL	≥ 5.7 或 8.0	数据存储
操作系统	Linux macOS Windows	推荐 Linux 生产环境
内存	≥ 1GB	服务运行 + 内存 Session
磁盘	视数据量而定	日志数据约每天 10 万条

5.2 部署拓扑





5.3 安装步骤

第一步：数据库准备

```
# 创建数据库并执行初始化 SQL
mysql -u root -p < migrations/init.sql
```

日志数据分表由程序自动创建，无需手动执行。

第二步：配置文件

```
cp config.yaml.example config.yaml
vim config.yaml # 根据实际环境修改
```

第三步：安装依赖

```
# 安装 Go (≥ 1.26)
# 如国内服务器，建议设置 Go 代理
go env -w GOPROXY=https://goproxy.cn,direct

# 安装 Node.js (≥ 18, 推荐使用 nvm 管理)
nvm install 24
nvm use 24
```

第四步：编译构建

方式一：使用 Makefile (推荐)

```
# 构建前端 (自动 npm install + npm run build)
make frontend-build

# 构建后端 (输出 bin/server 和 bin/cli)
make build
```

方式二：手动构建

```
# 构建前端
cd web && npm install && npm run build && cd ..

# 构建后端服务
go build -o bin/server ./cmd/server

# 构建 CLI 工具
go build -o bin/cli ./cmd/cli
```

第五步：启动服务

```
# 启动 Web 服务 (前端 SPA 已内嵌)
./bin/server config.yaml
```

首次启动时，系统会自动创建初始管理员账号 `admin`，初始密码将打印在控制台，请妥善记录：

```
=====
已创建初始管理员账号: admin
初始密码: xK7mN2pQ
请登录后立即修改密码
=====
```

5.4 配置文件说明

```
server:
  port: 8080                # 服务监听端口
  session_secret: "change-me-to-random-string" # Session 密钥 (必须修改为随机字符串)

database:
  host: "127.0.0.1"         # MySQL 地址
  port: 3306                # MySQL 端口
  user: "root"              # 数据库用户名
  password: ""              # 数据库密码
  dbname: "audit_platform"  # 数据库名称
```

```
charset: "utf8mb4"           # 字符集（勿修改）

password:
  salt: "ww1ocal-osslog"     # 密码加盐哈希的盐值（可自定义）

wecom:
  base_url: "https://your-wework-private-domain" # 企业微信私有化服务地址
  corpid: ""                 # 企业 ID
  corpsecret: ""            # 应用凭证密钥

log:
  level: "info"              # 日志级别（debug/info/warn/error）

labels:
  maintitle: "开放数据自动获取与查询小助手"      # 登录页主标题（可自定义）
  subtitle: "企业微信私有化开放数据Demo"        # 登录页副标题（可自定义）
```

安全提示：

- 务必修改 session_secret 为足够长的随机字符串
- 正确填写 wecom 下的 base_url、corpid、corpsecret
- 生产环境建议设置 password.salt 为自定义值

5.5 服务启动与健康检查

项目	地址
Web 管理界面	http://<服务器IP>:8080
健康检查端点	http://<服务器IP>:8080/health
API 基础路径	http://<服务器IP>:8080/api/

健康检查：

```
curl http://localhost:8080/health
# 返回: {"status":"ok"}
```

开发模式：

```
# 后端开发模式（直接 go run）
make dev

# 前端开发模式（Vite dev server，端口 5173）
make frontend-dev
```

5.6 CLI 运维工具

当管理员密码遗失或需要紧急操作时，可使用命令行工具：

```
# 重置 admin 用户密码（生成新随机密码，打印到控制台，状态回退为未激活）
./bin/cli reset-password

# 重置 admin 密码（使用自定义配置文件路径）
./bin/cli -c /path/to/config.yaml reset-password

# 禁用 admin 用户
./bin/cli disable-admin
```

6. API 端点一览

公开接口

方法	路径	说明
POST	/api/auth/login	用户登录
GET	/api/labels	获取页面标签配置
GET	/health	健康检查

登录后通用接口

方法	路径	说明
POST	/api/auth/logout	登出
GET	/api/auth/current	获取当前用户信息
POST	/api/auth/change-password	修改密码
GET	/api/operation-logs	查看操作日志
GET	/api/my-operation-logs	查看自己的操作日志

系统管理员接口（role=1）

方法	路径	说明
GET	/api/admins	管理员列表
POST	/api/admins	创建管理员
GET	/api/admins/:id	管理员详情
PUT	/api/admins/:id/disable	禁用管理员
PUT	/api/admins/:id/enable	启用管理员
DELETE	/api/admins/:id	删除管理员
PUT	/api/admins/:id/reset-password	重置密码
GET	/api/maintenance/config	获取维护配置
PUT	/api/maintenance/config	更新维护配置
POST	/api/maintenance/recount	触发数据重新统计
GET	/api/statistics	统计数据列表
GET	/api/statistics/grouped	按月分组统计
GET	/api/statistics/months	所有月份列表
GET	/api/all-operation-logs	全局操作日志
GET	/api/task/records-filter	任务记录查询（带过滤）

配置管理员接口（role=2）

方法	路径	说明
GET	/api/task/config	获取任务配置
PUT	/api/task/config	更新任务配置
PUT	/api/task/scheduled/start	启动定时任务
PUT	/api/task/scheduled/stop	停止定时任务
POST	/api/task/instant	创建即时任务
GET	/api/task/instant/progress	查询即时任务进度
GET	/api/task/instant/conflict	检查定时任务冲突
GET	/api/task/records	任务记录列表
GET	/api/private-keys	私钥列表
POST	/api/private-keys	创建私钥
PUT	/api/private-keys/:id	更新私钥
DELETE	/api/private-keys/:id	删除私钥

配置+审计管理员共享接口（role=2 或 role=3）

方法	路径	说明
GET	/api/data/feature-ids	数据项编码列表
GET	/api/data/query	查询日志数据（GET）
POST	/api/data/query	查询日志数据（POST，支持动态条件）
GET	/api/data/stats	统计数据（支持分钟/小时/天粒度）

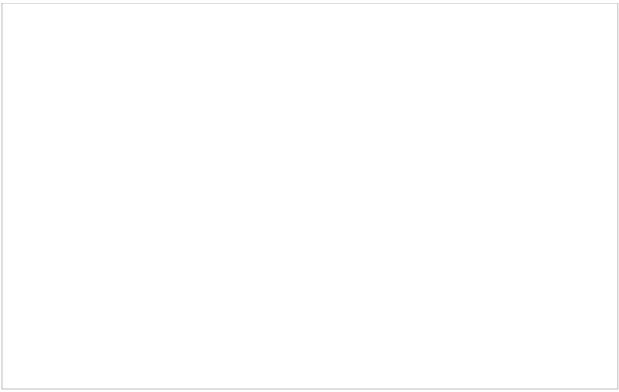
审计管理员接口（role=3）

方法	路径	说明
POST	/api/behavior-query	用户行为查询（跨数据类型）
GET	/api/behavior-query/validate-time	校验时间范围
POST	/api/intervention/revoke-message	撤回消息
POST	/api/intervention/delete-message	删除消息
POST	/api/intervention/delete-file	删除文件
POST	/api/intervention/dismiss-room	解散群聊
POST	/api/intervention/remove-members	移除群成员
POST	/api/intervention/get-members	获取群成员列表

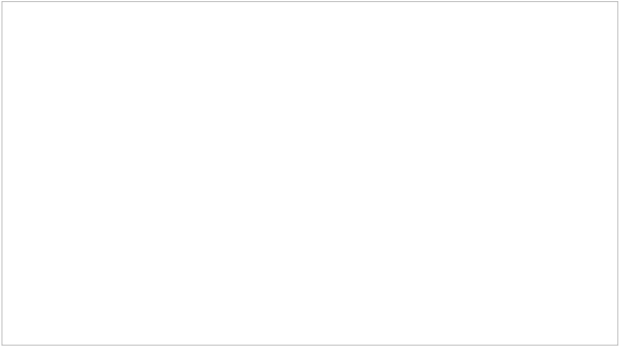
7. 前端界面预览

登录页

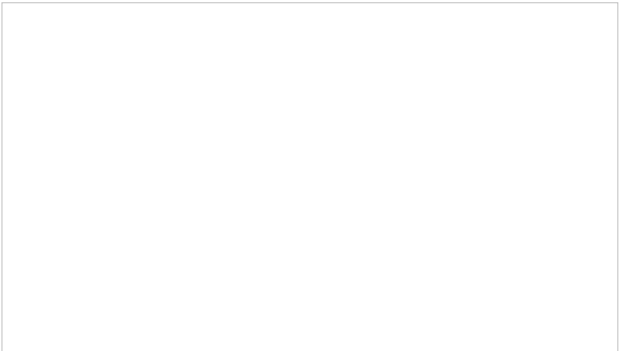
深蓝渐变背景 + 毛玻璃效果卡片，支持自定义标题和副标题。



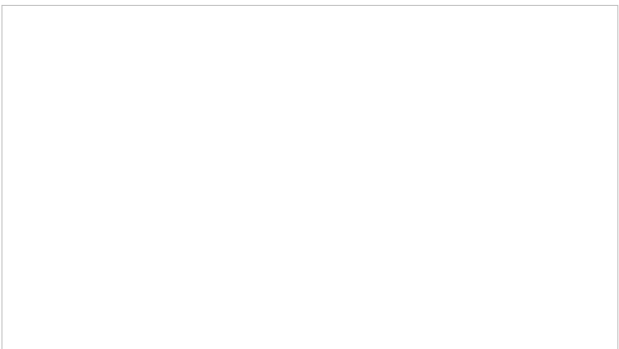
系统管理员 — 用户管理



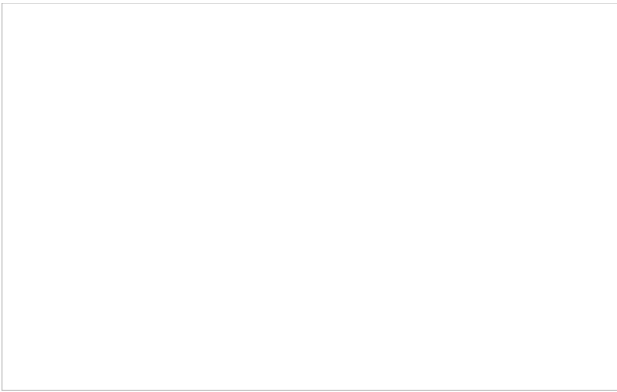
配置管理员 — 定时任务配置



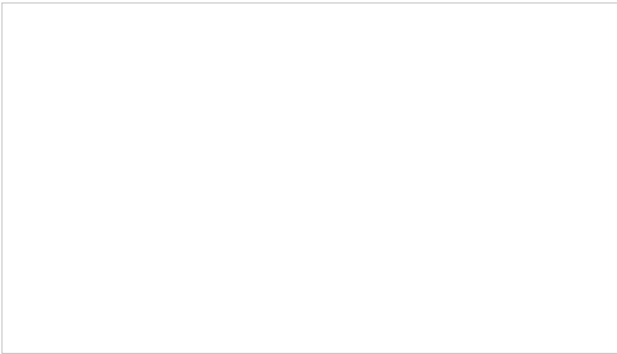
配置管理员 — 即时任务



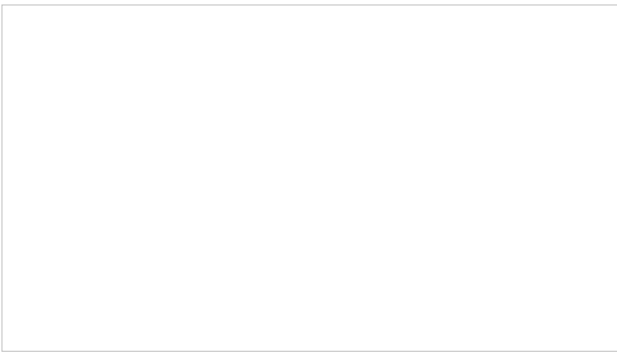
审计管理员 — 数据查询



审计管理员 — 行为查询



数据统计图表



8. 安全机制

安全维度	实现方式	说明
认证	Session + HttpOnly Cookie	每次请求通过 Cookie 携带 session_token，服务端内存 SessionStore 验证
鉴权	中间件级 RBAC	RequireRole / RequireAnyRole 在路由层拦截越权请求
密码存储	bcrypt + 自定义盐值	bcrypt(salt + password + salt), cost=10
密码复杂度	正则校验	≥8 字符，大小写字母 + 数字 + 特殊字符
防暴力破解	连续失败计数 + 时间锁	5 次失败锁定 10 分钟，锁定期间拒绝登录
首次登录强制改密	双密码哈希设计	未激活状态比对 init_password_hash，重置后清空
改密踢出	Session 级精确清除	DeleteByAdminID() 清除该用户所有活跃 Session
操作审计	全量操作日志	登录 配置变更 任务操作 用户管理 干预操作均记录
恒定时间比较	crypto/subtle.ConstantTimeCompare	防止计时攻击
Session 过期	24 小时 + 10 分钟清理	后台 goroutine 定期扫描过期 Session

9. 常见问题与排障

Q1: 首次启动忘记记录 admin 初始密码

使用 CLI 工具重置:

```
./bin/cli reset-password
```

Q2: Go 模块下载超时（国内服务器）

设置国内代理:

```
go env -w GOPROXY=https://goproxy.cn,direct
go env -w GONOSUMDB=*
```

Q3: 前端构建 npm 版本问题

```
cd web
rm -rf node_modules
npm install
cd .. && make frontend-build
```

Q4: 数据库连接失败

检查 config.yaml 中 database 配置，确认:

- MySQL 服务已启动
- 数据库 audit_platform 已创建
- 用户名密码正确
- 网络端口可达

Q5: 企微 API 返回错误

- 检查 config.yaml 中 wecom.base_url 是否正确（末尾不要有 /）
- 检查 corpid 和 corpsecret 是否有效
- 确认服务器可以访问企微私有化服务地址

Q6: 定时任务不执行

1. 确认任务状态为"运行中"
2. 检查是否配置了 featureID
3. 检查是否已添加对应版本的 RSA 私钥
4. 查看服务端日志是否有错误信息

Q7: 即时任务卡住

同一时间只能运行一个即时任务。如果上一个任务异常中断但状态未更新，可在数据库中手动将 task_records 中对应记录的 status 改为 2（失败）。